Project Outline: Digital Status

Your idea makes sense and solves a real problem: many people have no digital documents ready, so every time they apply for a job they have to recreate them. Your project is a **"professional digital vault (career vault) + shareable page + domain"**.

1. How a customer uses the service

1. They create an account (sign-up) and verify their email.
2. They choose a template (some are free, others are paid).
3. They pay (billing) and get space for their portfolio.
4. They fill in their information: CV, certificates, projects, video, contact details.
5. When applying for jobs, they share just a **link** (or their own domain) instead of recreating documents.
6. Recruiters visit, see a preview, and leave their details to download the CV.
7. The customer sees the messages and visitor details in their dashboard, **and only they can**.

2. The privacy principle: the super admin must not see visitor data

This is the key point and it needs careful thought, because it changes your first report. That report said the super admin sees a "Security Token Auditing Stream" containing recruiters' emails. That must be **removed**.

The truth you must accept: if the super admin has access to the database or the server, they can technically see everything unless the data is encrypted. So you need three layers:

a) Application-level access control

- The tables `recruiter_contacts` and `inquiries` are visible only to the portfolio's `owner_user_id`.
- No super admin route reads those tables.
- The super admin sees **numbers only** (for example "142 portfolios, 3,200 visitors"), not names, emails or phone numbers.

b) Field-level encryption of private data

- Store visitors' names, emails, phone numbers and messages **encrypted** (AES-256-GCM) before putting them into SQLite.
- Keep the customer's key separately, not alongside the database. That way, if someone steals `portfolio.sqlite`, the contents are unreadable.
- **Note:** if the key is derived from the customer's password (zero-knowledge), the super admin cannot read the data, but **a password reset would make the data unrecoverable**. That conflicts with your duty to "reset passwords". The better plan is a key stored separately on the server (envelope encryption), plus an honest statement in the Terms that "platform staff are not allowed to view your data, and all actions are logged".

c) Policy and law

- State in the Privacy Policy that the super admin does not view visitor data.
- Log every super admin action (audit log), visible to the customer.
- For serious situations (for example a legal complaint), have a written and publicly known procedure for accessing data.

3. How a recruiter accesses information

Step	What they see	What they provide
1. Opens the link	A gate page only (no private information)	Email
2. Confirms the OTP	A read-only preview of the portfolio	A 6-digit code
3. Wants to download the CV	A form	Name, email, position, phone, reason/description
4. Completes the form	The download	(Recorded in the customer's dashboard only)

Important: private data must be sent by the server **only after** the OTP is verified. It must not simply be hidden by the frontend.

4. Features by price

The difference must be real: Free works like a trial, and paid plans deliver real value.

Feature	Free	Professional	Enterprise
Templates	2 simple ones	All professional templates	Custom + Custom CSS
Link	`platform.com/name`	Their own domain	Multiple domains
Document storage	3 documents, 10 MB	20 documents, 200 MB	No strict limits
Pitch video	No	60 seconds	Many
OTP for visitors	No (open preview)	Yes	Yes + custom form
CV download form	No	Yes	Yes + extras
Seeing visitors	Count only	List and details	Detailed reports
Visitor messages	No	Yes	Yes + interview calendar
Data export	Yes	Yes	Yes
Platform badge	Shows “Made with…”	None	None

What to do when a customer stops paying (very important)

- **Do not block their data immediately.** Give a grace period (14 to 30 days).
- Their page becomes “paused” or falls back to the free level, but their data stays.
- They can **export** their data at any time. This is essential to the promise that “the portfolio is theirs”.

5. Super admin responsibilities

Allowed

- Reset passwords securely, through a link sent to the owner's email, without ever seeing the password.
- Resolve problems (support tickets), including suspending accounts used for abuse (phishing, scams).
- View overall figures (revenue, number of customers, subscription statuses).
- Manage templates and prices.
- Manage failed payments and refunds.
- Receive reports of service abuse.

Not allowed

- Viewing visitor data, messages, or customers' documents.
- Entering a customer's account without telling them (if necessary, use a recorded "impersonation" that the customer approves).

Inviting someone who has no account yet

If someone contacts the platform without signing up, the super admin can send them an **invitation link** (single-use, with an expiry time). When they click it, they create an account. This way the super admin never needs to see their private data.

6. Database additions

- `plans` and `plan_features`: features by tier, instead of putting them in code.
- `recruiter_contacts`: names, email, phone, position, reason (**encrypted**), plus `owner_user_id`.
- `inquiries`: messages sent to a portfolio.
- `invitations`: token, email, expires_at, used_at.
- `admin_audit_logs`: who, what, when, to whom.
- `documents`: customer documents (storage, size, type).
- On `admin_users`, add `role` (`customer` | `super_admin`).

7. How to build it (sequence)

1. **Accounts and login** done properly (hashing, rate limiting, email verification).
2. **Portfolio and templates** (free ones first).
3. **Documents + secure storage** (private storage, signed links).
4. **Recruiter gate** (OTP + form + data encryption).
5. **Customer dashboard** (visitors, messages, export).
6. **Plans + payments** (sandbox first, then live).
7. **Super admin** (figures, support, audit log).
8. **Custom domains + SSL** (the hardest step, so do it last).

8. Key business advice

- **Start small:** Free + Professional only. Add Enterprise once you have customers.
- **Privacy is what will set you apart** from other services such as Linktree. Put it first in your marketing: *"Your data is yours alone, and even the platform owners cannot see it."*
- Expect that some customers will want to **pay for a whole year** instead of monthly, so offer a discount.
- Rwanda: look at the payment methods in common use (MTN MoMo, Airtel Money), because most customers will use them more than cards.

9. Next step

Choose between a **full database schema** (all the new tables) or a **design of the customer and super admin dashboards**.

Part 2: System Workflow and Build Guide

This part is built on the Implementation Roadmap (v1.0, October 2026). It explains how the whole system works end to end, how the pieces connect, and exactly how each task gets built and tested.

10. System Overview: What Talks to What

Component	Technology	Responsibility
Frontend	React + Vite	Public portfolio pages, templates, owner dashboard, super admin panel, auth pages
API	Express	All business rules. The only thing that touches the database and private files
Database	SQLite (better-sqlite3), WAL mode, foreign keys ON	Users, portfolios, sections, plans, encrypted contacts
Private file storage	`private-documents/` folder	CVs and certificates. Never served statically
Email provider	Transactional email (SMTP or API)	Verification, OTP, resets, CV-downloaded alerts, renewal reminders
Payment provider	Sandbox first, then MTN MoMo / Airtel via a provider	Checkout and webhooks
Scheduler	node-cron	Grace periods, reminders, cleanup, backups
Edge and TLS	Caddy (Phase 9)	HTTPS for custom domains with on-demand certificates

The request pipeline

Every API request passes through the same chain, in this order: `helmet` → `cors` (locked to FRONTEND_URL) → `rate limit` → `load session` → `validate (zod)` → `auth` (who are you?) → `role` (what may you do?) → `plan guard` (does your plan allow it?) → `controller` → `service` → `database` → `central error handler`.

Two rules keep the code clean: controllers only translate HTTP into a service call and back, and services never touch `req` or `res`. That makes services easy to test without a server.

Four kinds of people and four trust zones

Who	Has an account?	Can reach	Cannot reach
Visitor	No	Public sections of a published portfolio, the OTP request form	Private sections, CV, any dashboard
Recruiter	No (verified by email OTP)	Private sections after OTP, CV download after the contact form	Other portfolios' private data, any dashboard
Owner (customer)	Yes, role = customer	Own portfolio, documents, leads, inquiries, billing, data export	Anyone else's data, any admin route
Super admin	Yes, role = super_admin	Metrics, account info, support, templates, plans, audit logs	Leads, inquiries, documents, encrypted fields

11. The Main Workflows, Step by Step

11.1 Signup to verified account (Phase 1)

1. User submits name, email, password to `POST /auth/signup`. Zod validates; the password is hashed with argon2 or bcrypt.
2. Server creates the `users` row with `email_verified = 0` and an `email_verifications` row holding a random token (stored hashed, expires in 24 hours).
3. Server emails a link containing the token. The user clicks it; the frontend calls `POST /auth/verify-email`.
4. Server marks the email verified and deletes the token. The user can now log in.
5. `POST /auth/login` checks the password, creates a `sessions` row (random token stored hashed) and sets an HttpOnly, Secure, SameSite cookie. Login is rate-limited per IP and per email.
6. Forgot password: `POST /auth/forgot-password` always answers success (never reveals whether an email exists), emails a single-use token; `POST /auth/reset-password` sets the new hash and deletes **all** sessions for that user.

7. The super admin account is created once at startup from `.env` values, never through signup.

11.2 Build and publish a portfolio (Phases 2 and 3)

1. After login the owner creates a portfolio: picks a slug (live availability check), a template, and gets `status = draft`.
2. The editor shows one form per section type (Hero, About, Experience, Education, Projects, Skills, Certificates, Contact). Each save writes one `portfolio_sections` row with `type`, `order`, `data_json`, `is_private`.
3. The owner marks which sections are private (shown only to verified recruiters). Everything else is public.
4. Before saving, the server validates the `data_json` shape per section type with zod, strips HTML from text, and allows only `https://` URLs.
5. The live preview renders the **same template component** the public page uses, with `mode = 'preview'`.
6. Template switch: the server checks the template's tier against the owner's plan, then updates `template_id`. Sections and data are untouched, so nothing is lost.
7. `PUT /portfolios/:id/publish` flips status to `published`. Drafts return 404 to the public.

11.3 Upload documents (Phase 4)

1. Frontend sends a multipart upload to `POST /documents`.
2. Middleware runs in order: auth → `checkLimit('documents')` (count and total MB from `plan_features`) → multer (type allowlist pdf/png/jpg, per-file size cap).
3. The server also checks the real file signature (magic bytes), not just the extension or the browser's MIME type.
4. The file is saved in `private-documents/` under a random name (for example a UUID). The original name is kept only in the database.
5. A `documents` row is created. To replace the CV, the owner sets `is_primary_cv`; the old file is deleted after the new one is safely stored.
6. The owner downloads their own files through an authenticated route that checks `owner_user_id`. A guessed URL returns 404, never 403, so file existence is not revealed.

11.4 The recruiter visit (Phase 5): the core of the product

Step	Recruiter does	Server does	Data released
1	Opens `/u/slug` or the custom domain	`GET /public/:slug` returns only sections where `is_private = 0`	Public sections only
2	Enters email, clicks Get access	`POST /recruiter/request-otp`: rate-limit per IP and per email, create a 6-digit code with a CSPRNG, store only its hash with a 10-minute expiry, email the code	Nothing
3	Types the code	`POST /recruiter/verify-otp`: constant-time compare, max 5 attempts then lock, on success create a `recruiter_sessions` row and set a short-lived HttpOnly cookie	Nothing yet
4	Page reloads private view	`GET /public/:slug/full` checks the recruiter cookie belongs to this portfolio, then returns private sections	Private sections
5	Clicks Download CV	Frontend shows the contact form because the session has no completed contact	Nothing
6	Submits name, email, position, phone, reason	`POST /recruiter/contact`: validate, encrypt every field, store in `recruiter_contacts` with `owner_user_id`, mark the session as contact-completed	Download unlocked
7	Receives the file	`GET /public/:slug/download` checks a short-lived signed link, streams the file, and emails the owner: Someone downloaded your CV	CV file

Recruiter session states: `none` → `otp_requested` → `verified` → `contact_completed`. The server only moves forward one state at a time and checks the state on every route. Skipping a step (for example calling `/download` right after OTP) returns 403.

Failure rules: wrong OTP five times locks that email on that portfolio for a cool-down period; an expired code needs a new request; the cookie expires after a short time (for example 2 hours) and the recruiter simply repeats OTP; the cookie is only valid for the portfolio it was issued for.

The most important test in the whole project: load the public page and read every network response and the HTML source before OTP. No private section, no CV URL and no recruiter data may appear anywhere.

11.5 Owner dashboard (Phase 6)

1. Overview: view count, OTP verifications, CV downloads, new inquiries (counts are plan-aware: Free sees numbers only, Professional sees the list).
2. Leads: the server reads `recruiter_contacts` where `owner_user_id = current user`, decrypts in memory, and returns them. Nothing is cached in decrypted form.
3. Inquiries inbox: messages from the public contact form, stored encrypted, shown with read/unread state.
4. Share box: link with copy button and QR code, generated client-side from the public URL.
5. Exports: leads as CSV, and a full export of the user's own data (profile, sections, document list, leads, inquiries) as a ZIP. Export is allowed on every plan, even when paused.
6. Cross-owner test: owner B and the super admin both get 403 on owner A's leads and inquiries endpoints.

11.6 Subscription lifecycle (Phase 7)

State	Meaning	Page	Features	Triggered by
free	No paid plan	Live	Free limits	Signup
active	Paid and current	Live	Plan limits	Verified payment webhook
grace	Expired, within 14 days	Live, owner sees a banner	Plan limits	Cron at period end
paused	Grace ended, unpaid	Shows a paused notice	Dashboard and export still work	Cron after 14 days

State	Meaning	Page	Features	Triggered by
cancelled	Owner cancelled	Live until period end, then free	Falls back to free limits	Owner action

Checkout flow: owner picks a plan → `POST /billing/checkout` → the **server** looks up the price from `plans` and creates a pending `payments` row → the provider redirects or prompts on the phone → the provider calls `POST /billing/webhook` → the server verifies the signature, checks the event id has not been processed (idempotency), marks the payment paid, and activates or extends the subscription. The frontend never decides that a payment succeeded. Reminders go out 7 days and 1 day before renewal, and again when grace starts. **Nothing is ever deleted because of non-payment.**

11.7 Super admin workflows (Phase 8)

1. **Metrics:** counts and revenue only, computed with aggregate queries that never select contact or message columns.
2. **Password reset:** admin clicks Send reset on a user; the server emails a normal reset token to the user. The admin never sees a password or a token.
3. **Invitation:** admin enters an email; the server creates a single-use, expiring token in `invitations` and emails the link; signup via that link marks it used.
4. **Suspend / reactivate:** sets a flag on the user; a suspended user's page returns a notice and the user sees the reason and an appeal route.
5. **Support tickets and abuse reports:** a report links to a portfolio slug; the admin can unpublish the page, but can read only public content.
6. **Audit:** every admin action writes `admin_audit_logs` (actor, action, target user, time). The affected user sees these entries in their own dashboard.
7. **Impersonation** is off by default. If it is ever added, it requires the owner's explicit consent, a time limit, and a visible audit record.

11.8 Custom domain flow (Phase 9)

1. Owner enters `mycv.com`; the server creates a `custom_domains` row with `status = pending` and a random `verification_token`.
2. The dashboard shows a DNS TXT record to add (for example name `_digitalstatus` with the token as value) plus the A or CNAME record that points to the server.

3. Owner clicks Verify; the server looks up the TXT record with Node DNS. If it matches, `status = verified`, `verified_at` is set.
4. When the first HTTPS request arrives, Caddy calls the **ask** endpoint (`/internal/domain-check?domain=...`). The endpoint answers 200 only for verified domains, otherwise 404, and is reachable only from localhost. Caddy then obtains the certificate.
5. A host-header middleware maps the `Host` to a portfolio and rejects unknown hosts. The request then follows the same public-page path as the slug URL.
6. If the plan expires or the domain is removed, the row is deactivated and the ask endpoint stops approving it.

12. Data Model at a Glance

Table	Key columns	Who can read it
users	id, email, password_hash, role, email_verified, suspended_at	Owner (own row), admin (account info only, no hash)
sessions	id, user_id, token_hash, expires_at	Server only
email_verifications, password_resets	user_id, token_hash, expires_at, used_at	Server only
user_keys	user_id, wrapped_data_key, key_version	Server only
portfolios	id, owner_user_id, slug, template_id, theme_json, status	Owner; public if published
portfolio_sections	portfolio_id, type, sort_order, data_json, is_private	Owner; public only if is_private = 0
documents	id, owner_user_id, type, original_name, stored_name, size, mime, is_primary_cv	Owner only
recruiter_otps	portfolio_id, email_hash, code_hash, expires_at, attempts, locked_until	Server only

Table	Key columns	Who can read it
recruiter_sessions	id, portfolio_id, email_hash, state, expires_at	Server only
recruiter_contacts	id, owner_user_id, portfolio_id, name_enc, email_enc, position_enc, phone_enc, reason_enc	Owner only (decrypted on read)
inquiries	id, owner_user_id, portfolio_id, sender_enc, message_enc, read_at	Owner only
plans, plan_features	plan_id, feature_key, value (limit or boolean)	Public read of plan info
subscriptions	user_id, plan_id, state, period_end, grace_end	Owner; admin (status only)
payments	id, user_id, amount, currency, provider_ref, provider_event_id (unique), status	Owner; admin (status and amount)
custom_domains	portfolio_id, domain, verification_token, status, verified_at	Owner; server
invitations	token_hash, email, expires_at, used_at	Admin
support_tickets	id, user_id, subject, status	Owner; admin
admin_audit_logs	id, admin_id, action, target_user_id, created_at	Affected user; admin

General rules: every table has `id`, `created_at`, `updated_at`; foreign keys use ON DELETE rules chosen deliberately; the `*_enc` columns hold ciphertext only; tokens and OTPs are stored hashed, never in plain text.

13. API Map

Area	Endpoint	Who	Notes
Auth	POST /auth/signup, /verify-email, /login, /logout, /forgot-password, /reset-password	Public	Rate-limited, zod-validated
Auth	GET /me	Logged in	Returns role, plan, limits
Portfolio	GET and POST /portfolios; GET, PUT, DELETE /portfolios/:id	Owner	Ownership check on every id
Portfolio	PUT /portfolios/:id/sections/:sectionId, PUT /portfolios/:id/publish	Owner	Server-side tier check for templates
Portfolio	GET /slugs/check?slug=	Logged in	Reserved words blocked
Documents	POST /documents, GET /documents, DELETE /documents/:id, PUT /documents/:id/primary, GET /documents/:id/download	Owner	checkLimit('documents') on upload
Public	GET /public/:slug	Anyone	Public sections only
Public	GET /public/:slug/full	Verified recruiter	Private sections
Public	GET /public/:slug/download	Recruiter with completed contact	Short-lived signed link, streamed
Public	POST /public/:slug/inquiries	Anyone	Encrypted, rate-limited, spam check
Recruiter	POST /recruiter/request-otp, /verify-otp, /contact	Anyone	requireFeature('otp_gate') for owners on paid plans
Dashboard	GET /leads, /leads/export.csv, /inquiries, /stats, /me/export	Owner	Owner-only; plan decides the detail level

Area	Endpoint	Who	Notes
Billing	GET /plans, GET /billing/subscription, POST /billing/checkout	Owner	Server computes price
Billing	POST /billing/webhook	Provider	Signature check, idempotent
Admin	GET /admin/metrics, /admin/users, /admin/tickets, /admin/audit-logs	Super admin	No route returns contacts, inquiries or documents
Admin	POST /admin/users/:id/suspend, /reactivate, /password-reset, POST /admin/invitations	Super admin	Each writes an audit log
Domains	GET, POST, DELETE /domains, POST /domains/:id/verify	Owner	Enterprise or Professional per plan
Internal	GET /internal/domain-check	Caddy only	Localhost-restricted
Ops	GET /health	Anyone	Used by uptime monitoring

14. Security Design Details

Sessions. Random 32-byte session token in an HttpOnly, Secure, SameSite=Lax cookie; only its hash is stored. Because cookies are used, state-changing routes also check the `Origin` header (or a CSRF token) so another site cannot trigger them.

Encryption (envelope encryption). One `MASTER_KEY` lives outside the database (environment or a secrets manager). Each owner gets a random data key, stored in `user_keys` encrypted (wrapped) by the master key. Private fields are encrypted with the owner's data key using AES-256-GCM, a fresh random 12-byte IV per field, and the stored value carries the key version, IV, auth tag and ciphertext. Stealing `portfolio.sqlite` alone gives unreadable data.

Why not derive the key from the user's password. It would make the data readable only by the owner, but a password reset would destroy the data, and the roadmap requires admin-triggered resets. Envelope encryption keeps resets safe.

Be honest about the limit. Someone with both database and server environment access could technically decrypt. The promise to customers is therefore enforced by three layers together: no admin route reads that data, the data is encrypted at rest, and every admin action is logged and visible to the user. Say exactly that in the Terms and Privacy Policy, and do not claim more.

Recruiter OTP. Use `crypto.randomInt` for the code; store HMAC of the code with a server secret; compare with `timingSafeEqual`; count attempts per code; lock after five; delete the code on success.

Signed download links. The link carries document id, recruiter session id and expiry, signed with HMAC; it is valid for a few minutes and checked again on the server before streaming.

Uploads. Allowlist of types, magic-byte check, size cap, random stored name, storage outside the web root, `Content-Disposition: attachment`, `X-Content-Type-Options: nosniff`.

Input and output. Zod on every endpoint, text sanitized on save, only `https://` links, theme tokens validated (hex colours, font allowlist), React escapes output by default, and a Content-Security-Policy is added in Phase 10.

Admin boundary as code. Admin routes use a dedicated database module that exposes only aggregate and account-info queries. The encrypted tables are not importable there, so the rule is enforced by structure and not just by discipline. Automated tests prove it.

15. How Every Task Gets Built: The Repeatable Recipe

Every feature in every phase follows the same nine steps. This keeps work predictable and makes it easy to see what is left.

1. **Migration.** Add a numbered SQL file (for example `007_recruiter_contacts.sql`). Never edit an old migration after it has run anywhere.
2. **Zod schema.** Define the exact shape of the request and the response in the module's `schema.js`.
3. **Service function.** Write the business logic in `service.js`, with the database calls inside it.
4. **Route and middleware.** Wire `routes.js`: validate → auth → role → plan guard → controller.

5. **Test.** Write the happy path, the wrong-user path (403 or 404), and the bad-input path before moving on.
6. **Frontend API function.** Add one function in `frontend/src/api/` that calls the endpoint. Pages never call `fetch` directly.
7. **Hook and page.** Wrap the call in a hook (loading, error, data), then build the page or component.
8. **Check the phase's Done-when line.** Run it by hand as a real user would.
9. **Tick the box in TODO.md** and commit with a message that names the phase.

Backend module layout

Each folder in `backend/src/modules/` has the same files: `routes.js` (URLs and middleware), `controller.js` (HTTP in and out), `service.js` (business rules), `schema.js` (zod), and `module.test.js`. Shared helpers live in `utils/` (`crypto`, `tokens`, `sanitize`) and shared guards live in `middleware/` (`auth`, `role`, `planLimit`, `rateLimit`, `validate`).

Frontend layout

Pages are grouped by audience (`public`, `dashboard`, `admin`, `auth`). All portfolio rendering goes through `templates/registry.js` and shared section components, so a new template is only layout and style.

16. Phase-by-Phase Task Breakdown

Tasks are listed in the order they should be done inside each phase. The durations come from the roadmap.

Phase 0: Foundation (2 to 3 days)

- Create the repo, the backend and frontend folders, and `.gitignore` (including `.env` and `private-documents/`).
- Write `config/env.js` that reads and validates every environment variable at startup and refuses to boot if one is missing.
- Open SQLite with `PRAGMA foreign_keys = ON` and `journal_mode = WAL`; write the migration runner that applies files in order and records them in a `migrations` table.

- Add `helmet`, `cors`, `express-rate-limit`, the zod `validate` middleware and the central error handler.
- Add `GET /health`; fix the Windows-friendly port-in-use message in `server.js`.
- Frontend: Vite app, router, an empty layout, and a base API client that sends cookies.

Phase 1: Accounts and Auth (1 week)

- Migrations: users, sessions, email_verifications, password_resets.
- Build `utils/tokens` (random token plus hash), the mail config with a development fallback that prints emails to the console, and the `auth` and `role` middleware.
- Endpoints in this order: signup → verify-email → login → logout → /me → forgot-password → reset-password. Seed the super admin from `.env`.
- Frontend: signup, verify, login, forgot, reset pages and a protected-route wrapper that checks role.
- Tests: duplicate email, wrong password lockout, reused token rejected, sessions invalid after reset, customer gets 403 on an admin route.

Phase 2: Portfolio Core and Editor (1 to 1.5 weeks)

- Migrations: portfolios, portfolio_sections. Write `utils/sanitize` and a zod schema per section type.
- Endpoints: portfolio CRUD, section save and reorder, slug check (with a reserved-words list such as admin, api, login), publish and unpublish.
- Frontend: dashboard shell, one editor form per section, a live preview, an autosave with a visible Saved indicator.
- Tests: owner A cannot edit owner B's portfolio, javascript: URLs rejected, drafts return 404 publicly.

Phase 3: Templates (1.5 weeks)

- Define the data contract (`profile`, `sections`, `theme`, `mode`, `gate`) as a documented type.
- Build shared section components first: Hero, About, Experience, Education, Projects, Skills, Certificates, VideoPitch, Contact, DownloadCV.
- Build `minimal_grid` completely, then the CSS-variable theme system with validated tokens, then `registry.js`.

- Build `clean_resume` next. Only then start the paid templates `tech_pro` and `creative`.
- Server: on save, reject a template whose tier is above the user's plan.
- Tests: switching templates loses no data; empty sections are hidden; the page looks right at phone width.

Phase 4: Documents and Storage (1 week)

- Migration: documents. Create `private-documents/` outside any static path.
- Multer config, magic-byte check, random file names, size and count limits read from `plan_features` (hardcode the free values temporarily until Phase 7).
- Endpoints: upload, list, delete, replace primary CV, owner-only download.
- Frontend: upload area with progress and error messages, document list, replace-CV button.
- Tests: a guessed file URL returns 404; a renamed .exe is rejected; the limit blocks the fourth free document.

Phase 5: Public Page and Recruiter Gate (1.5 weeks)

- Migrations: recruiter_otps, recruiter_sessions, recruiter_contacts, user_keys. Build `utils/crypto` (encrypt, decrypt, wrap key) with unit tests first.
- Build in this order: `GET /public/:slug` → request-otp → verify-otp → `/full` → `/contact` → signed `/download` → owner email alert.
- Frontend: the shared gate overlay (email step, code step, contact form step), then the DownloadCV button states.
- Tests: no private data before OTP, five wrong codes lock, expired code fails, download fails without the form, a signed link expires.

Phase 6: Owner Dashboard (1 week)

- Migration: inquiries, plus view and download counters.
- Endpoints: leads list, inquiries list and mark-read, stats, CSV export, full data export, public inquiry form endpoint.
- Frontend: Overview, Leads, Inquiries, Share (copy button and QR code), Export.
- Tests: owner B and the admin get 403 on owner A's leads and inquiries; CSV contains decrypted values only for the owner.

- **Stop here and test with 5 to 10 real people.** Change the product based on what they do, not on what they say.

Phase 7: Plans and Billing (1.5 weeks)

- Migrations: plans, plan_features, subscriptions, payments. Seed Free and Professional first; Enterprise later.
- Replace the temporary hardcoded limits with `requireFeature(key)` and `checkLimit(key)` that read `plan_features`.
- Pricing page, checkout, webhook with signature check and a unique `provider_event_id`, then the node-cron jobs (period end → grace → paused) and renewal reminders.
- Frontend: upgrade prompts at every limit, a billing page with payment history.
- Tests: a tampered price in the request is ignored; the same webhook delivered twice only extends the plan once; a paused page keeps all data and export still works.

Phase 8: Super Admin (1 week)

- Admin dashboard (counts and revenue), user list, suspend and reactivate, password-reset trigger, invitations, tickets, abuse reports, template and plan management.
- `admin_audit_logs` written by a single helper that every admin action must call; show the log to the affected user.
- Tests: iterate over every `/admin` route and assert that none returns contacts, inquiries or documents; assert every admin write creates an audit row.

Phase 9: Custom Domains and Production (1.5 to 2 weeks)

- Migration: custom_domains. Build add, verify (DNS TXT), remove and the ask endpoint.
- Host-header middleware; Caddy config with on-demand TLS and the ask URL; pm2 process file; VPS setup with a firewall.
- Backups: scheduled `VACUUM INTO` to a dated file, copied off the server, and a documented restore procedure.
- Logging and uptime monitoring.
- Test with a real domain pointed at the server.

Phase 10: Hardening and Launch (1 week)

- Terms of Service, Privacy Policy and a recruiter consent line on the contact form.

- SPF, DKIM and DMARC for the sending domain; move email to a transactional provider.
- CSP headers, a security checklist pass, upload and XSS tests, report-abuse flow, error tracking.
- Restore a backup onto a clean machine to prove it works.

17. Testing Plan

These tests protect the promises the product makes. Write each one in the phase shown, and run all of them before every deploy.

Test	Phase	Must happen
Customer calls an admin route	1	403
Reset password invalidates old sessions	1	Old cookie rejected
Owner B edits owner A's portfolio	2	403 or 404
Draft portfolio requested publicly	2	404
Free user saves a Pro template by crafted request	3	Rejected
Guessed URL to a stored document	4	404
Public page and network responses before OTP	5	No private data anywhere
Six wrong OTP attempts	5	Locked after the fifth
Download without completed contact form	5	403
Owner B or admin reads owner A's leads	6	403
Webhook sent twice	7	Processed once
Client sends a lower price	7	Ignored; server price used
Expired plan	7	Page paused, data intact, export works
Every admin route scanned	8	No contacts, inquiries or documents returned

Test	Phase	Must happen
Admin action without audit row	8	Not possible
Ask endpoint with an unverified domain	9	404
Restore backup on a clean machine	10	App starts with the data

18. Configuration and Background Jobs

Environment variables

Variable	Purpose
NODE_ENV, PORT, FRONTEND_URL, PUBLIC_BASE_URL	Basic runtime and CORS lock
DATABASE_PATH, PRIVATE_DOCS_PATH	Where SQLite and private files live
SESSION_SECRET, OTP_HMAC_SECRET, DOWNLOAD_SIGNING_SECRET	Separate secrets for sessions, OTP hashing and signed links
MASTER_KEY	Wraps each owner's data key. Back it up separately and never commit it
SUPER_ADMIN_EMAIL, SUPER_ADMIN_PASSWORD	Seeds the first admin
MAIL_*	Email provider settings
PAYMENT_*	Provider keys and webhook secret

Losing `MASTER_KEY` means losing every encrypted lead and message, so store a copy in a safe place away from the server and the database backups.

Scheduled jobs (node-cron)

Job	Frequency	What it does
Cleanup	Hourly	Deletes expired OTPs, recruiter sessions, verification and reset tokens

Job	Frequency	What it does
Subscription check	Daily	Moves active → grace → paused using the dates in `subscriptions`
Renewal reminders	Daily	Emails owners 7 days and 1 day before renewal and when grace starts
Backup	Daily	`VACUUM INTO` a dated file, then copies off the server
Domain recheck	Weekly	Re-verifies TXT records and deactivates domains that no longer pass

19. Decisions to Settle and Gaps Found

The two documents are close, but these points differ or are still open. Decide them before the phase that depends on them.

Topic	What the documents say	Suggested decision
Order: pay first or free first	Part 1 says the customer pays before filling in data; the roadmap says ship the free tier first	Follow the roadmap: free tier first, billing in Phase 7
User table name	Part 1 says add role to `admin_users`; the roadmap uses one `users` table with a role column	One `users` table with a `role` column
Gate on the free plan	Part 1 says Free has an open preview and no contact form; Phase 5 builds the gate for everyone	Gate is on for everyone in the MVP; in Phase 7 it becomes a paid feature through `requireFeature('otp_gate')`. Decide whether Free users can offer a CV download at all, and if so whether it is open
Video pitch	Pro gets 60 seconds; no storage plan	For the MVP allow only an embed link from an allowlist (YouTube or Vimeo), which avoids video hosting costs

Topic	What the documents say	Suggested decision
Link previews	The public page is a React single-page app	WhatsApp and LinkedIn previews need server-injected title and image tags per portfolio; add this before launch
Public URL	`platform.com/izina` in Part 1, `/u/slug` in the roadmap	Use `/u/slug` so slugs never collide with app routes
Currency and pricing	Not set	Price in RWF, offer an annual discount, set after Phase 6 feedback
Data protection	Roadmap mentions Rwanda's data-protection law	Have a local lawyer review the Privacy Policy and the recruiter consent wording; the law is Law No. 058/2021 on personal data and privacy, which should be confirmed
Email provider	Not chosen	Pick one with good deliverability in East Africa before Phase 1 goes live

20. Working Routine and TODO.md Starter

The rhythm

1. Pick the next unchecked task in the current phase.
2. Follow the nine-step recipe in section 15.
3. Run the whole test suite, then the phase's Done-when check.
4. Tick the box, commit, and write one line in a short progress log: what was finished, what is blocked.
5. Never start a phase while the previous one has an unticked Done-when line.

TODO.md starter

- [ ] **Phase 0** Foundation: migration runner, WAL and foreign keys, helmet/cors/rate limit/zod, health route, .env ignored
- [ ] **Phase 1** Auth: tables, signup, verify, login, logout, forgot/reset, super admin seed, 403 test
- [ ] **Phase 2** Portfolio: tables, section forms, preview, slug check, publish, sanitize

- [ ] **Phase 3** Templates: shared sections, minimal_grid, theme tokens, registry, clean_resume, then Pro templates
- [ ] **Phase 4** Documents: multer, private storage, limits, replace CV, 404 test
- [ ] **Phase 5** Recruiter gate: crypto utils, OTP, session, /full, contact form, signed download, owner alert, no-leak test
- [ ] **Phase 6** Dashboard: leads, inquiries, stats, exports, share box, 403 tests, then user testing with 5 to 10 people
- [ ] **Phase 7** Billing: plans, limits, checkout, webhook, cron, reminders, upgrade prompts
- [ ] **Phase 8** Super admin: metrics, users, invitations, tickets, audit log, route-scan test
- [ ] **Phase 9** Domains and production: TXT verify, Caddy ask, host middleware, VPS, backups, monitoring
- [ ] **Phase 10** Launch: legal pages, SPF/DKIM/DMARC, CSP, abuse flow, restore test

Before building any feature

Check it against the five objectives at the start of the roadmap: stop rewriting documents, owner controls their data, recruiter gate, enforced paid tiers, limited super admin. If it serves none of them, postpone it.